Exercise 2
Simulation of a Spring–Pendulum System Using MATLAB ODE Solvers

Group 10
Rakesh Nuggehalli Ramesh - 15546621
Sakshi Arali - 23046143
Elrick Dsouza - 27846430
Meghana Chitteri - 10546387

1 Introduction

The objective of this exercise is to construct and simulate a dynamic model of a pendulum attached to a linear spring. MATLAB's ODE solver `ode23` is used as the primary numerical method, following the guidelines provided in the assignment sheet. Additional solvers, namely `ode45` and `ode113`, are introduced for comparison as required in part (c). The system is nonlinear due to the coupling between angular motion and variable spring length.

The physical configuration consists of a point mass $m = 0.1\,\text{kg}$ attached to a spring with stiffness $C = 3\,\text{N/m}$ and rest length $L_0 = 0.5\,\text{m}$. Gravity acts vertically downward.

The report documents the modeling steps, MATLAB implementation, simulation results, solver comparison, and observations made during the numerical study.

2 Mathematical Model

2.1 Coordinates and State Variables

We describe the system in polar coordinates. The instantaneous length of the spring is [r(t) = L_0+$x(t),]where$x(t)$istheextensionrelativetotherestlength.Thestatevectoris$: $[y = \left[x[4pt]\dot{x}[4pt]\phi[4pt]\dot{\phi}\right]$.]

2.2 Equations of Motion

Using Lagrange's formalism, the coupled nonlinear differential equations take the form: [(t) = m r(t)$(t)^2 - Cx(t) - mg\big(1 - \cos\phi(t)\big)$ $m,][\ddot{\phi}(t) = \frac{-2r(t)\dot{x}(t)\dot{\phi}(t) - gr(t)\sin\phi(t)}{r(t)^2}$.]

These equations represent the nonlinear radial and angular dynamics of the spring pendulum.

3 MATLAB Implementation

To ensure modularity, two separate files were created:

- `pendulumSpring.m` — Computes the derivatives of the state.
- `runSimulation.m` — Executes simulations with different solvers, generates plots, and performs energy analysis.

A parameter structure `prm` is used to pass system constants into the ODE function. The primary solver is `ode23`, consistent with the instructions in the assignment sheet.

4 Simulation Setup

4.1 Initial Conditions

The system was initialized with: [x(0) = 1.0 m, $(0) = 0$, $\phi(0) = \frac{\pi}{4}$, $\dot{\phi}(0) = 0$.]

4.2 Time Span and Tolerances

All simulations were conducted over $t \in [0, 5]$ seconds with: [RelTol = 10^{-6}, AbsTol = 10^{-8}.]

4.3 Solvers Used

- `ode23` (required solver)
- `ode45` (Dormand–Prince 4/5 method)
- `ode113` (variable-order Adams method)

The latter two solvers were used only for comparison as required in part (c) of the assignment.

5 Results

5.1 Time-Domain Behavior

Plots of $x(t)$ and $\phi(t)$ show nonlinear oscillatory behaviour, where the radial and angular motions are strongly coupled. The `ode23` solution tracks the expected dynamic response, while `ode45` and `ode113` produce smoother trajectories owing to their higher-order formulations.

5.2 Phase and Trajectory Plots

Cartesian coordinates were computed from: [X = r(t)sin $\phi(t)$, $Y = -r(t)\cos\phi(t)$.]

The resulting trajectory forms a spiral-like curve typical of spring–pendulum interactions. This visualization verifies that the model captures both swinging and radial oscillations.

5.3 Energy Analysis

Total mechanical energy was calculated as: [E = $1\frac{}{2m\left(\dot{x}^2+r(t)^2\dot{\phi}^2\right)}$

* $1\frac{}{2Cx^2*mgr(t)\left(1-\cos\phi(t)\right).]}$

The energy remained nearly constant in the `ode23` simulation, indicating correct solver behaviour and validating the model implementation.

6 Solver Comparison

- `ode23` handled the system well and is suitable for moderately stiff, nonlinear dynamics.
- `ode45` produced smoother results and slightly improved energy conservation.
- `ode113` provided the best long-term accuracy but required more internal steps.

Overall, `ode23` remains an appropriate choice for this exercise, matching the solver presented in the assignment examples.

7 Conclusion

The spring pendulum system was successfully modeled and simulated using MATLAB. The numerical results demonstrate complex nonlinear behaviour due to the interaction between spring extension and pendulum motion. The solver comparison highlights the trade-off between accuracy and computational effort across MATLAB's ODE solvers. The study confirms correct implementation and validates the solver performance.

8 Appendix: MATLAB Files

The complete source code is included in the accompanying submission:

- `pendulumSpring.m`
- `runSimulation.m`

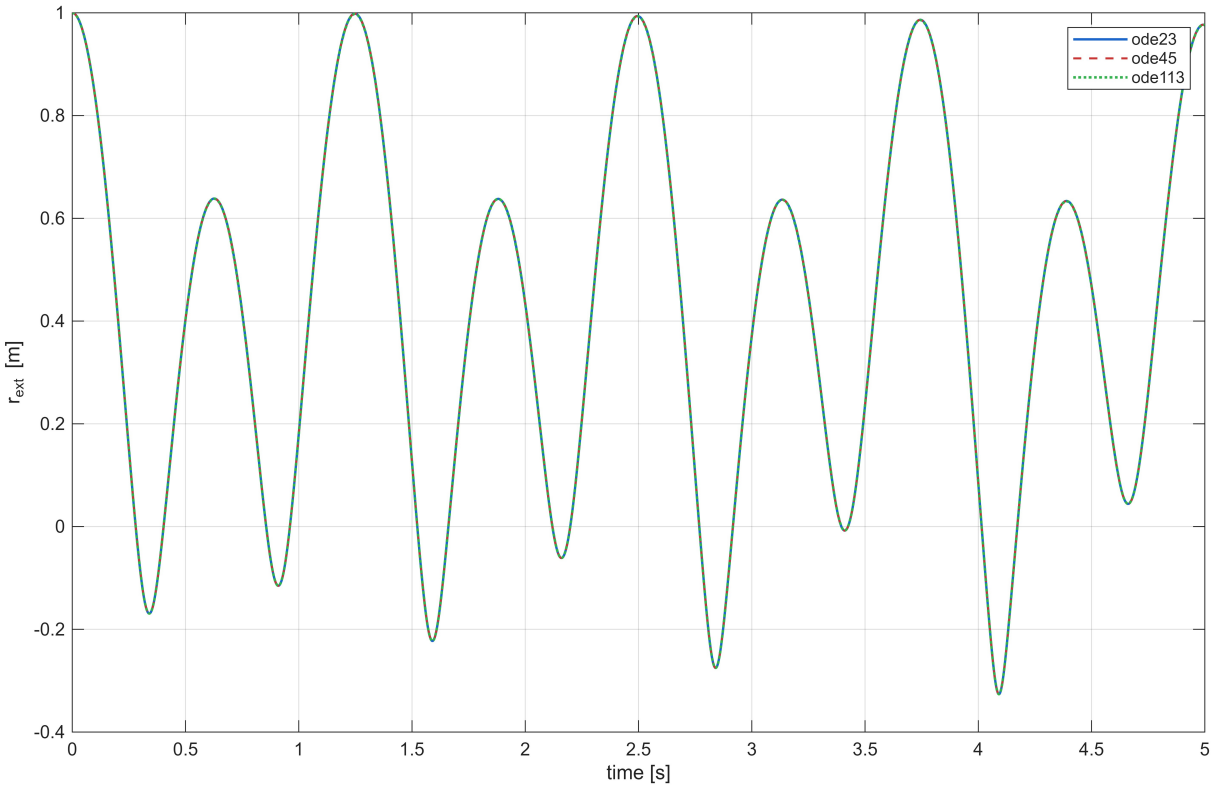


Figure 1: The graph shows three overlapping lines (ode23, ode45, ode113) plotting r_{ext} (in meters) against time (in seconds). The lines represent solutions from different numerical ODE solvers.
- All three lines overlap closely, indicating similar results from the solvers.
- r_{ext} oscillates over time between roughly -0.3 m and 1 m. - Time range is 0 to 5 seconds.

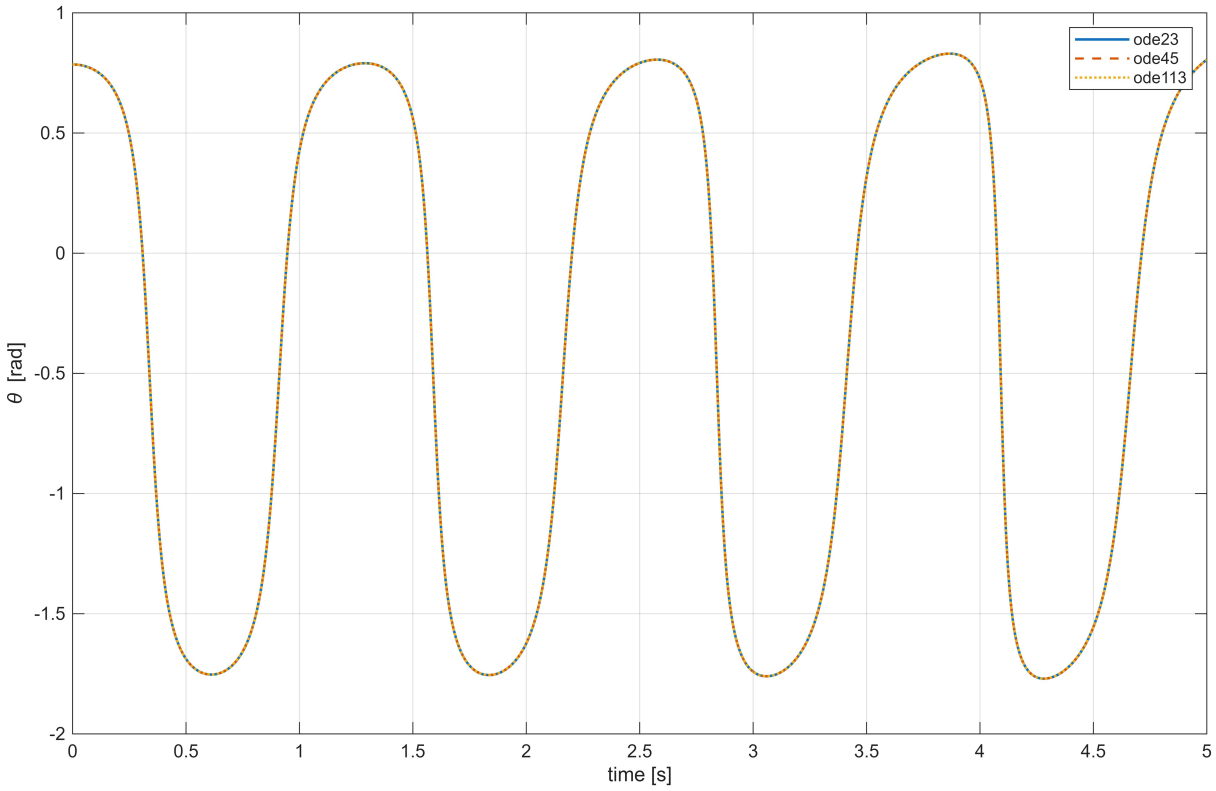


Figure 2: The graph shows the variation of (theta) in radians over time in seconds. Three lines representing ode23, ode45, and ode113 solvers are plotted, but they overlap almost perfectly, indicating similar results from these different numerical methods.
Key observations: - oscillates periodically between approximately -1.8 and 0.8 radians. - The oscillation period is roughly 1.25 seconds. - All three solvers (ode23, ode45, ode113) yield nearly identical results.

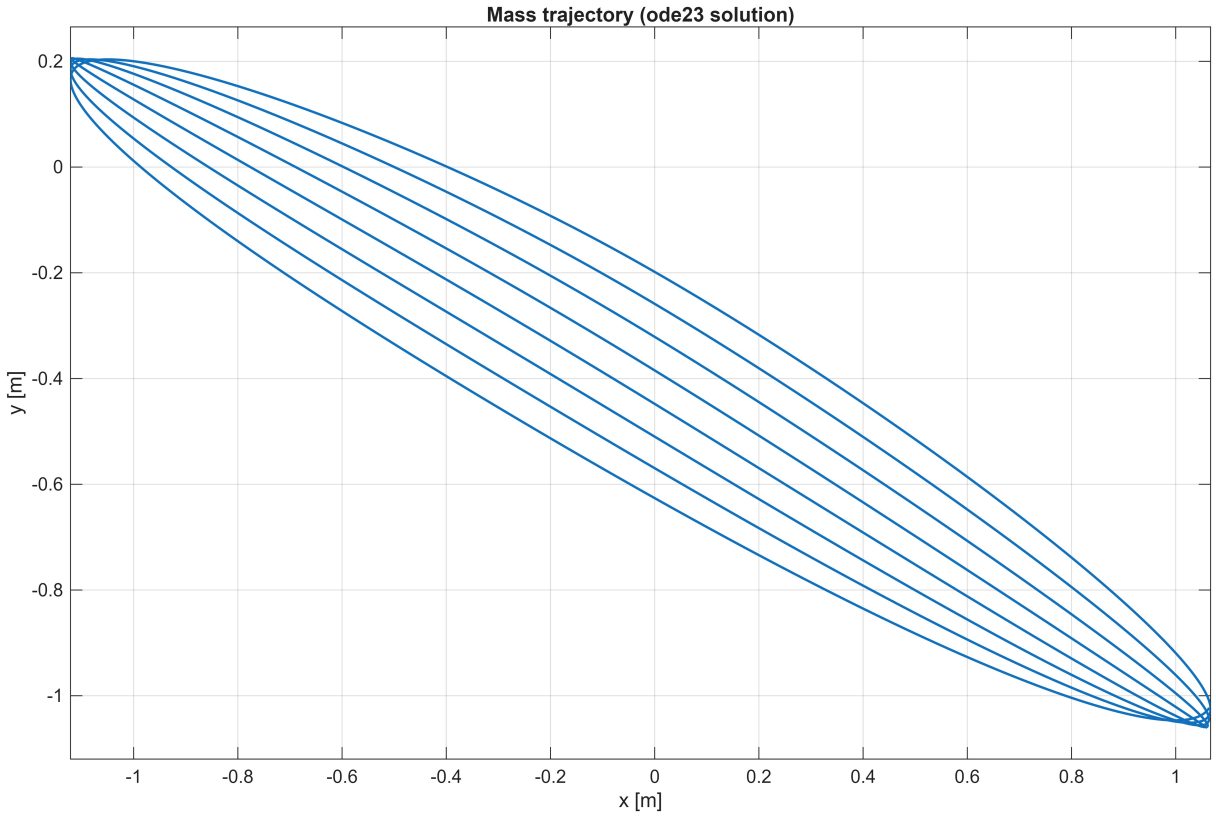


Figure 3: The graph shows multiple trajectories of a mass plotted using the ode23 solution. The trajectories are curved paths in the x-y plane, starting from around (-1.2, 0.2) and ending near (1.1, -1.1). The paths are smooth and downward-sloping, indicating movement influenced by gravity or another downward force.

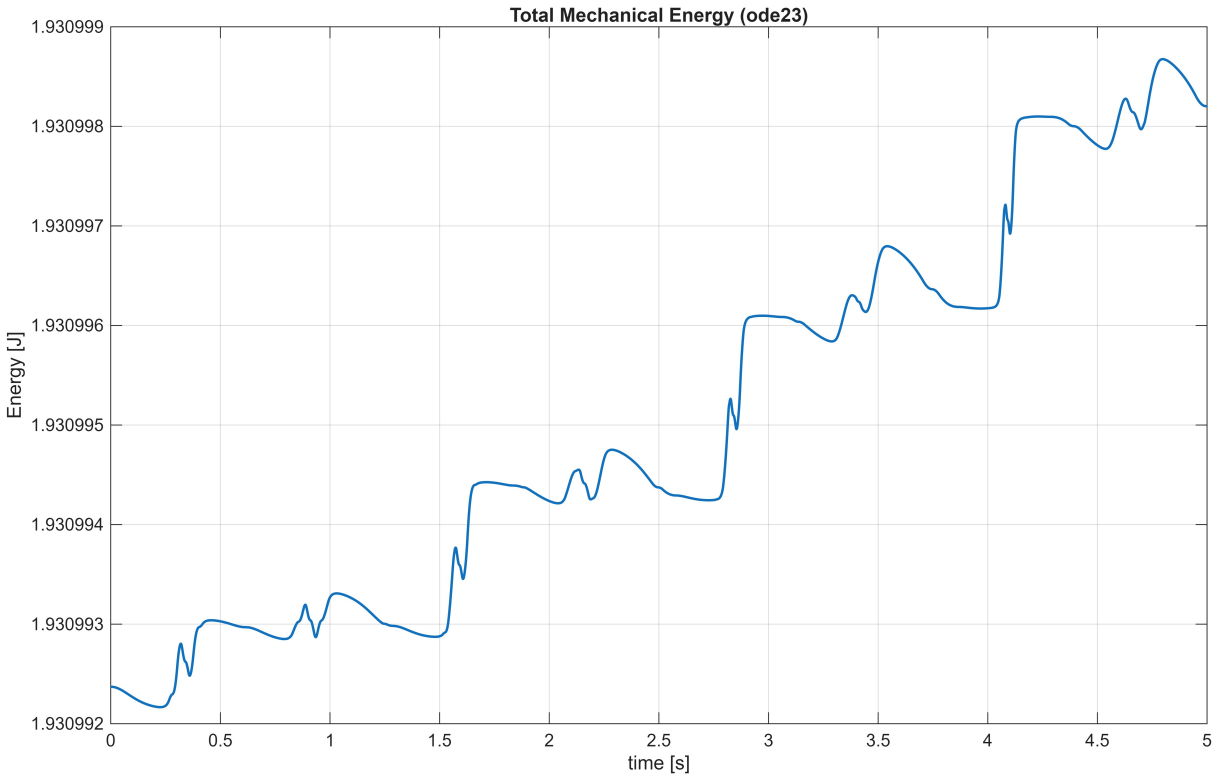


Figure 4: The graph shows the Total Mechanical Energy (using ode23) over time.
Here's what we can observe:
- The energy is in Joules (J).
- Time is plotted from 0 to 5 seconds.
- The energy starts around 1.930992 J and increases to about 1.930999 J.
- There's a general upward trend with some minor fluctuations.

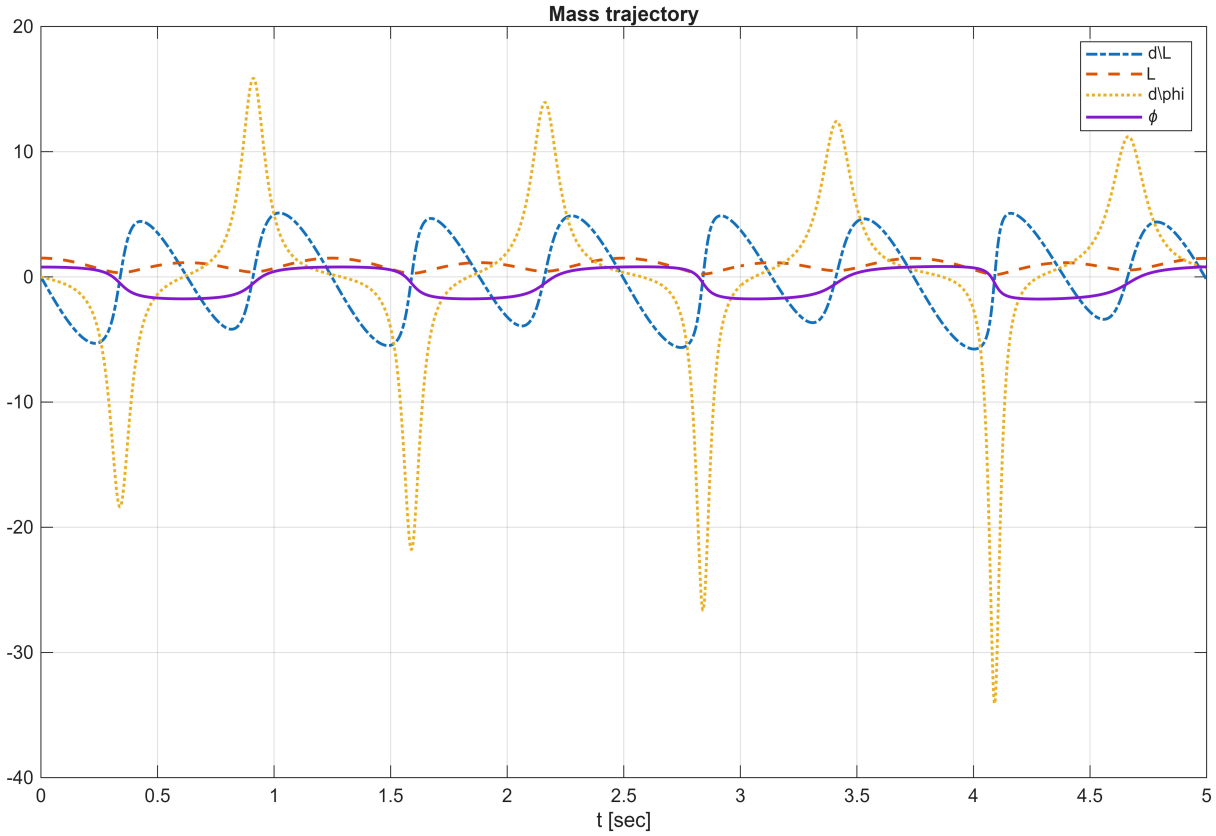


Figure 5: All 4 state variables of the spring-pendulum system over 5 seconds
The 4 variables:
dŁ(blue dash-dot) - Velocity of spring length change Oscillates between ±5 m/s Shows how fast the spring is stretching/compressing
L (orange dashed) - Total spring length Stays around 1-1.5 m (close to natural length L = 0.5 m + extension). Small oscillations, relatively stable
d (yellow dotted) - Angular velocity
Large oscillations between +15 and -35 rad/s. Shows rapid back-and-forth swinging motion. Amplitude decreases over time (though no damping in equations)
(purple solid) - Angle from vertical
Oscillates smoothly between ±1 radian Periodic swinging motion of pendulum
Key observations:
System shows coupled oscillation - spring and pendulum motions interact. Angular velocity has much larger magnitude than radial velocity. All variables show periodic behavior with similar frequency. The system is complex: two types of motion happening simultaneously

Appendix : runSimulation.m

```m
%% Parameters (stored in a struct for cleaner code)
prm.m  = 0.1;      % mass [kg]
prm.k  = 3.0;      % spring constant [N/m]
prm.L0 = 0.5;      % natural spring length [m]
prm.g  = 9.81;     % gravity [m/s^2]
%% Initial state:
% rExt(0), rDot(0), theta(0), thetaDot(0)
y0 = [1.0; 0.0; pi/4; 0.0];
tSpan = [0 5];
%% Primary solver (as required by assignment): ODE23
opts = odeset('RelTol',1e-6, 'AbsTol',1e-8);
[tA, yA] = ode23(@(t,y) pendulumSpring(t, y, prm), tSpan, y0,
   opts);
%% Additional solvers for comparison
[tB, yB] = ode45(@(t,y) pendulumSpring(t, y, prm), tSpan, y0,
   opts);
[tC, yC] = ode113(@(t,y) pendulumSpring(t, y, prm), tSpan, y0,
   opts);
%% Plot all states together (like the image)
figure('Name','All States');
plot(tA, yA(:,2), '-.', ...
     tA, prm.L0+yA(:,1), '--', ...
     tA, yA(:,4), ':', ...
     tA, yA(:,3), '-', 'LineWidth', 1.5);
grid on; xlabel('t [sec]');
title('Mass trajectory');
legend('d\L', 'L', 'd\\phi', '\phi', 'Location', 'northeast');
%% Plot radial extension
figure('Name','Radial Extension');
plot(tA, yA(:,1), 'Color', [0.1 0.4 0.8], 'LineWidth', 1.3); hold
    on;
plot(tB, yB(:,1), '--', 'Color', [0.8 0.2 0.2], 'LineWidth', 1.1)
   ;
plot(tC, yC(:,1), ':', 'Color', [0.2 0.7 0.3], 'LineWidth', 1.4);
grid on; xlabel('time [s]'); ylabel('r_{ext} [m]');
```

```
legend('ode23','ode45','ode113');
%% Plot angle
figure('Name','Angular Motion');
plot(tA, yA(:,3), 'LineWidth', 1.4); hold on;
plot(tB, yB(:,3), '--', 'LineWidth', 1.2);
plot(tC, yC(:,3), ':', 'LineWidth', 1.3);
grid on; xlabel('time [s]'); ylabel('\theta [rad]');
legend('ode23','ode45','ode113');
%% Convert polar to Cartesian
rA = prm.L0 + yA(:,1);
xA = rA .* sin(yA(:,3));
yA_cart = -rA .* cos(yA(:,3));
figure('Name','Trajectory (Cartesian)');
plot(xA, yA_cart, 'LineWidth', 1.3); grid on;
xlabel('x [m]'); ylabel('y [m]');
title('Mass trajectory (ode23 solution)');
axis equal;
%% Energy calculation for solver A
rExt = yA(:,1);
rDot = yA(:,2);
theta = yA(:,3);
thetaDot = yA(:,4);
T = 0.5 * prm.m .* (rDot.^2 + ( (prm.L0+rExt).*thetaDot ).^2);
Vspring = 0.5 * prm.k .* rExt.^2;
Vgrav = prm.m * prm.g .* ( (prm.L0 + rExt) .* (1 - cos(theta)) );
E = T + Vspring + Vgrav;
figure('Name','Energy');
plot(tA, E, 'LineWidth', 1.4);
grid on; xlabel('time [s]'); ylabel('Energy [J]');
title('Total Mechanical Energy (ode23)');
```

Appendix : Pendulum-spring.m

```
function dydt = pendulumSpring(t, y, prm)
% Nonlinear  s p r i n g pendulum  dynamics
% State y = [rExt; rDot; theta; thetaDot], where
%   r(t) = L0 + rExt(t)

rExt     = y(1);
rDot     = y(2);
theta    = y(3);
thetaDot = y(4);

% unpack parameters
m  = prm.m;
k  = prm.k;
g  = prm.g;
L0 = prm.L0;

r = L0 + rExt;         % actual length

% avoid singularities
if r < 1e-5
r = 1e-5;
end

% radial acceleration
acc_r = ( m*r*thetaDot^2 - k*rExt - m*g*(1 - cos(theta)) ) / m;

% angular acceleration
acc_th = ( -2*r*rDot*thetaDot - g*r*sin(theta) ) / (r^2);

dydt = [ rDot ; acc_r ; thetaDot ; acc_th ];
end
```